

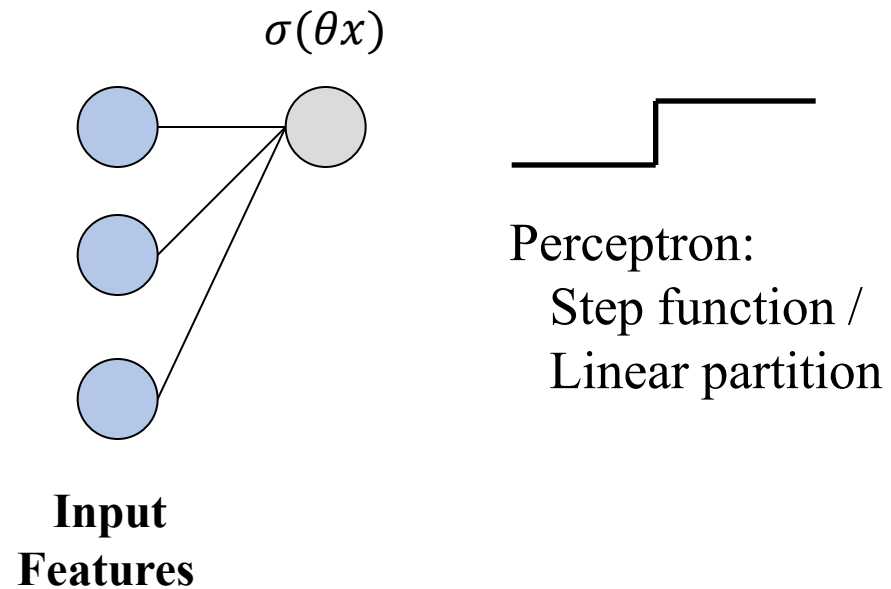
Neural Networks

Adopted from slides by Alexander Ihler and Andrew Ng

Representation

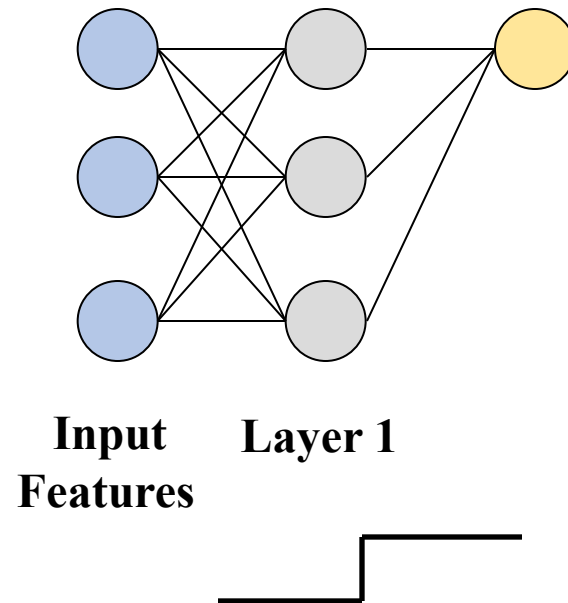
Features of MLPs

- Simple building blocks
 - Each element is just a perceptron f'n
- Can build upwards



Features of MLPs

- Simple building blocks
 - Each element is just a perceptron f'n
- Can build upwards



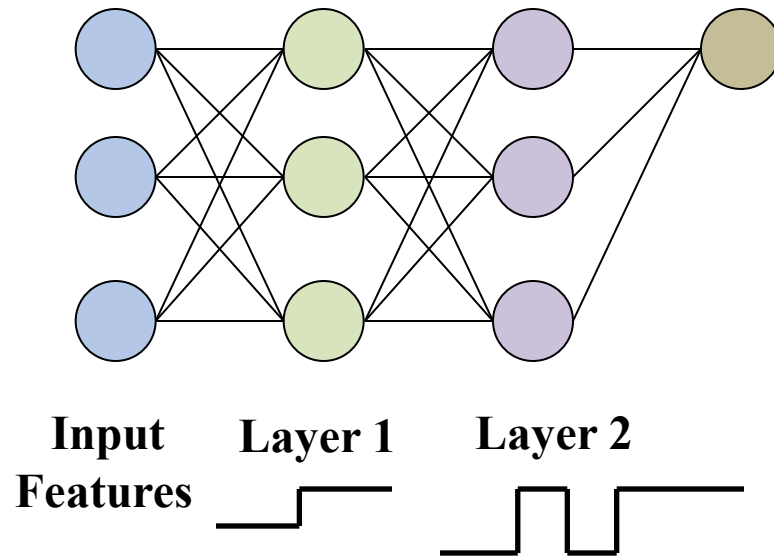
2-layer:

“Features” are now partitions

All linear combinations of those partitions

Features of MLPs

- Simple building blocks
 - Each element is just a perceptron f'n
- Can build upwards

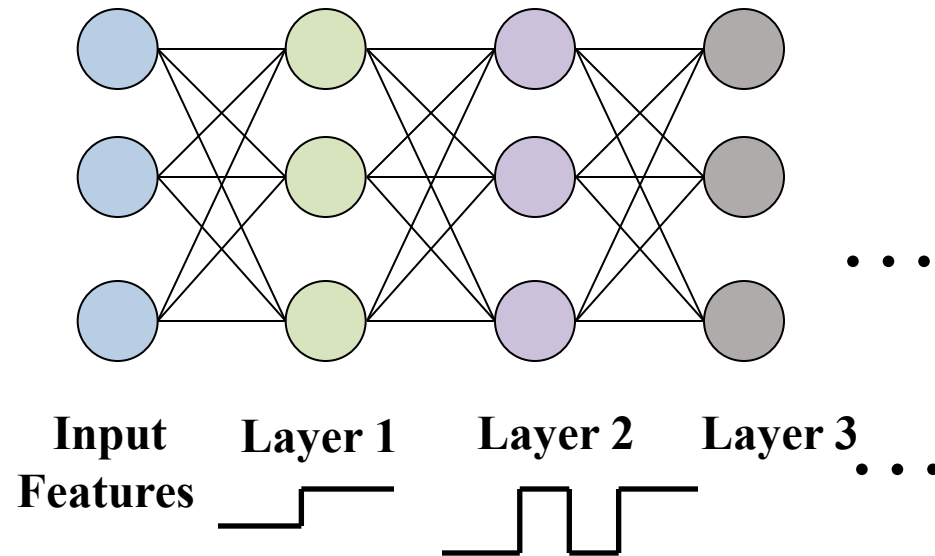


3-layer:

“Features” are now complex functions
Output any linear combination of those

Features of MLPs

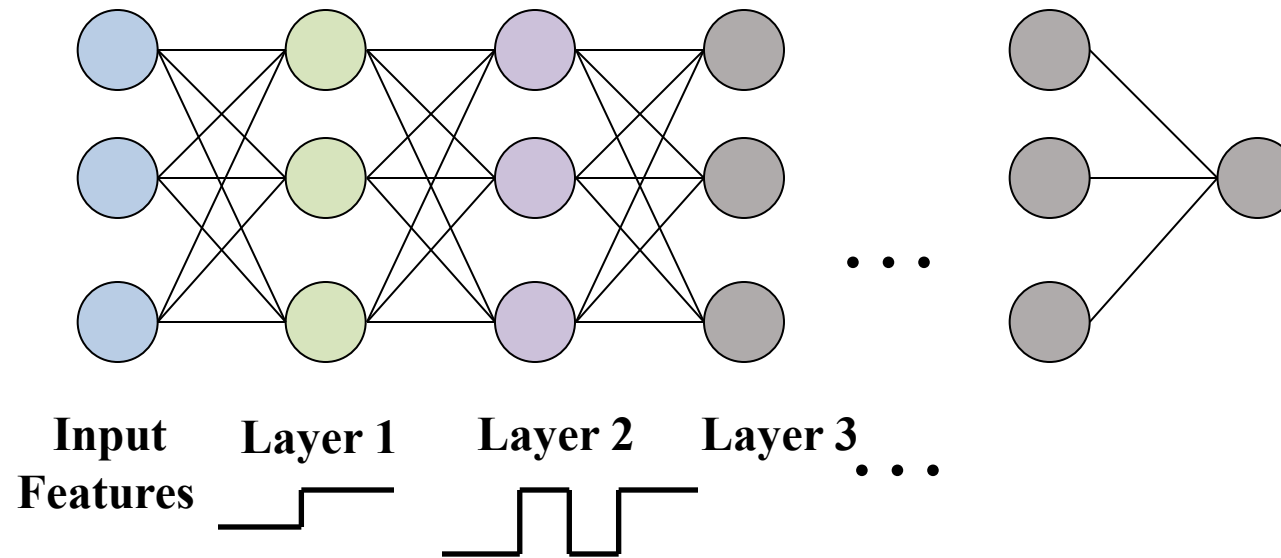
- Simple building blocks
 - Each element is just a perceptron f'n
- Can build upwards



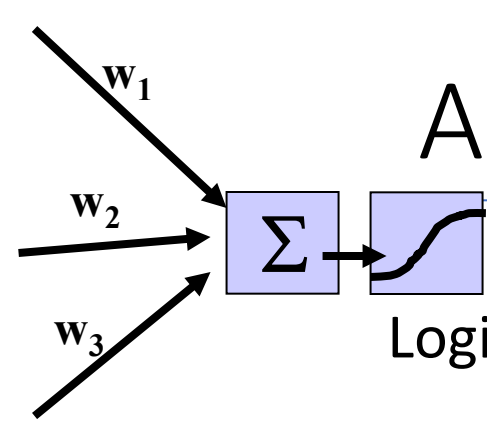
Current research:
“Deep” architectures
(many layers)

Features of MLPs

- Simple building blocks
 - Each element is just a perceptron f'n
- Can build upwards

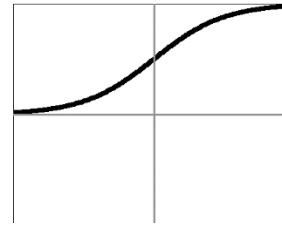


Activation functions



Logistic

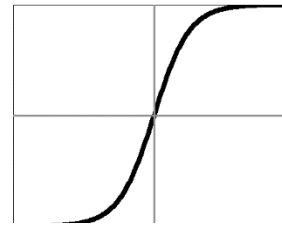
$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$



$$\frac{\partial \sigma}{\partial z}(z) = \sigma(z)(1 - \sigma(z))$$

Hyperbolic
Tangent

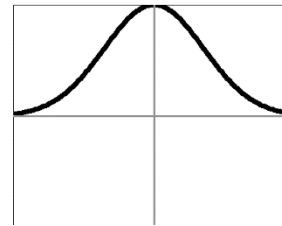
$$\sigma(z) = \frac{1 - \exp(-2z)}{1 + \exp(-2z)}$$



$$\frac{\partial \sigma}{\partial z}(z) = 1 - (\sigma(z))^2$$

Gaussian

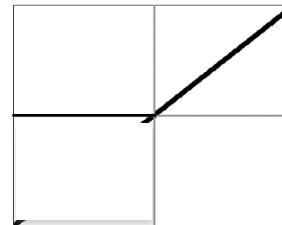
$$\sigma(z) = \exp(-z^2/2)$$



$$\frac{\partial \sigma}{\partial z}(z) = -z\sigma(z)$$

ReLU
(rectified linear)

$$\sigma(z) = \max(0, z)$$



$$\frac{\partial \sigma}{\partial z}(z) = \mathbb{1}[z > 0]$$

Linear

$$\sigma(z) = z$$

and many others...

Optimization – Backpropagation

Optimization - gradient descent

```
Initialize  $W$   
Do {  
     $W \leftarrow W - \alpha \nabla J(W)$   
} while (stop condition)
```

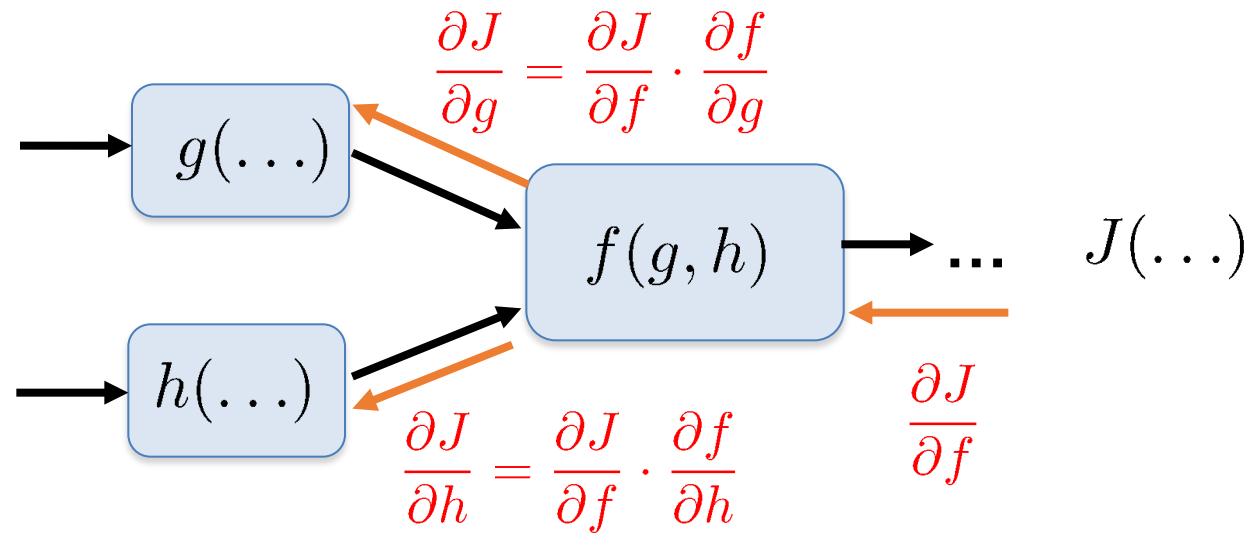
- Compute $\frac{\partial J(W)}{\partial w_{j,i}^l}$
- Different from logistic regression, where all parameters are equivalent in terms of positions in the loss function, parameters in neural network are different due to their locations in the network structure.

Gradient calculations

- Chain rule:

- If $F(x) = f(g(x))$, then $F'(x) = f'(g(x))g'(x)$

- Alternative form, if z depends on y , and y depends on x , then $\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$



Ex: $f(g, h) = g^2 h$

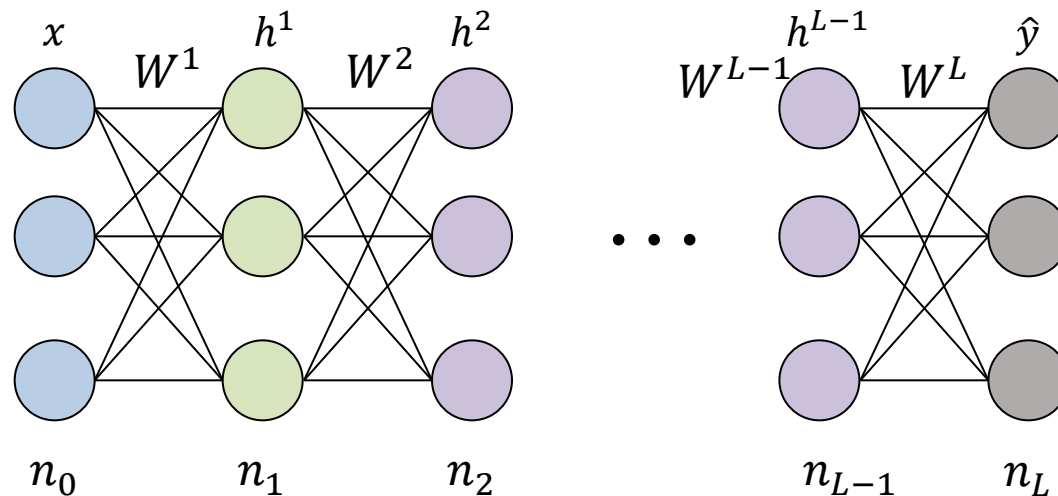
$$\frac{\partial J}{\partial g} = \frac{\partial J}{\partial f} \cdot 2g(\cdot)h(\cdot)$$

$$\frac{\partial J}{\partial h} = \frac{\partial J}{\partial f} \cdot g^2(\cdot)$$

save & reuse info (g, h) from forward computation!

Forward and back propagation

- $x = h^0$
- $z^l = W^l \cdot h^{l-1}$
- $h^l = \sigma(z^l)$
- $\hat{y} = \sigma(z^L), \mathcal{L}(y, \hat{y})$
- $\beta^L = \mathcal{L}'(y, \hat{y}) * \sigma'(z^L)$
- $\beta^l = (\beta^{l+1} \cdot W^{l+1}) * \sigma'(z^l)$
- $\nabla_{W^l} J = (\beta^l)^T \cdot h^{l-1}$



Training a neural network

1. Randomly initialize weights to small values
2. Implement forward propagation to get $h_w(x^{(i)})$ for any $x^{(i)}$
3. Implement code to compute loss function $J(w)$
4. Implement backprop to compute partial derivatives $\frac{\partial J}{\partial w_{j,i}^l}$

for $i=1:m$

 Perform forward propagation and backpropagation
 using example $(x^{(i)}, y^{(i)})$

Training a neural network

5. Use gradient descent or advanced optimization method with backpropagation to try to minimize $J(w)$

PyTorch Example Code

- Define a neural network

```
import torch.nn as nn
import torch.nn.functional as F

class TwoLayerNet(nn.Module):
    def __init__(self, D_in, H, D_out):
        Super(TwoLayerNet, self).__init__()
        self.linear1 = nn.Linear(D_in, H)
        self.linear2 = nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = F.relu(self.linear1(x))
        y_pred = self.linear2(h_relu)
        return y_pred

D_in, H, D_out = 100, 50, 10
model = TwoLayerNet(D_in, H, D_out)
```

PyTorch Example Code

- Define the loss function and optimizer

```
import torch.optim as optim

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr = 0.01)
```

PyTorch Example Code

- Train the neural network

```
output_batch = model(train_batch)      # compute model output
loss = loss_fn(output_batch, labels_batch) # calculate loss
optimizer.zero_grad()                  # clear previous gradients
loss.backward()                         # compute gradients of all variables wrt loss
optimizer.step()                       # perform updates using calculated gradients
```